Graph Algorithms

Shortest path

Readings

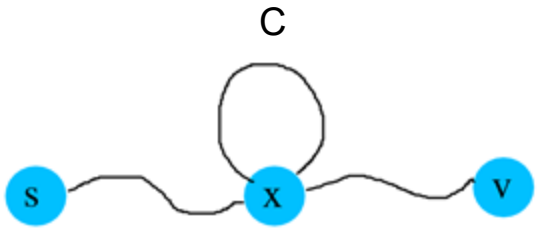
- Chapter 24 of CLR
  - Shortes-path
  - Dijkstra
  - Bellman-ford

https://jeffe.cs.illinois.edu/teaching/algorithms/book/08-sssp.pdf

Bellman-Ford Algorithm

Bellman-ford Algorithm

- If G has no negative cycles, then there exists a shortest path from s to any node u that uses at most n-1 edges.
- **Proof.** Suppose there exists a shortest path from s to u consisting of n or more edges
  - A path of length at least n must visit at least n+1 nodes
  - There exists a node x that is repeated (pigeonhole principle) → There is a cycle C
  - Can remove C without increasing cost of path



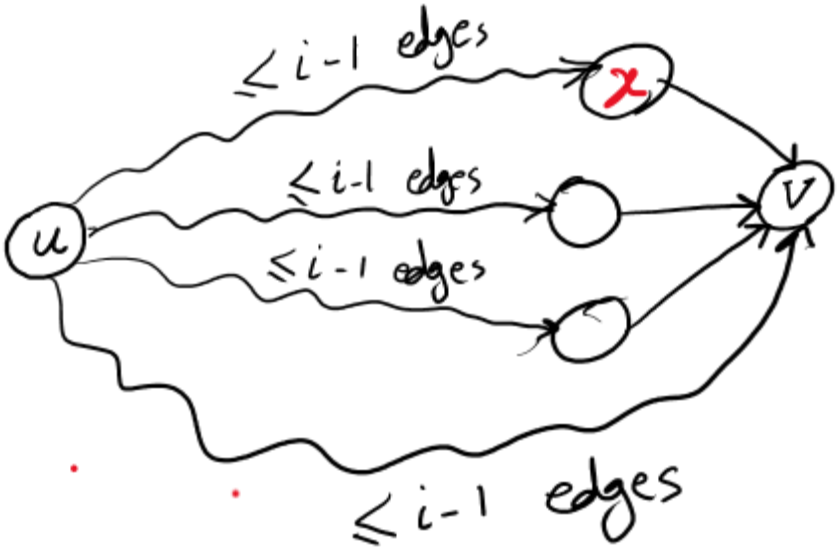
Bellman-ford Algorithm

Recall: If a graph $G = (V, E)$ contains a negative- weight cycle, then some shortest paths may not exist.

Bellman-Ford algorithm: Finds the shortest-path lengths from a ***source*** s in V to all v in V or determines that a negative-weight cycle exists.

Bellman-ford Algorithm: dynamic programming approach

- The shortest path problem has the optimal substructure property:
  - All subpaths of a shortest path are shortest paths.
- Can we solve the problem using dynamic programming?
- Can we solve the problem recursively?
- What is the subproblem?
- First let’s focus on finding the length (weight) of the shortest path



Shortest path from u to v with at most i edges is the minimum of the followings:

Shortest path with at most i-1 edges from u to v(subproblem)

Shortest path with at most i-1 edges from u to x (a vertex pointing to v) + weight of edge (x,v)

Bellman-ford Algorithm: dynamic programming approach

D(i,v) = weight of a shortest path from **s** to **v** that uses at most **i** edges

- Goal D(n-1, v) for each v
  - If there is no negative cycle, then there exists a shortest path that is simple

$$D(i,v) = min \begin{cases} D(i-1,v) & \text{shortest path uses at most i-1 edges} \\ min_{(u,v)\in E}\{D(i-1,u) + w(u,v)\} & \text{shortest path uses exactly i edges} \end{cases}$$

$$D(0,s) = 0$$

$$D(0,v) = \infty \text{ where } v \neq s$$

Bellman-ford Algorithm: dynamic programming approach

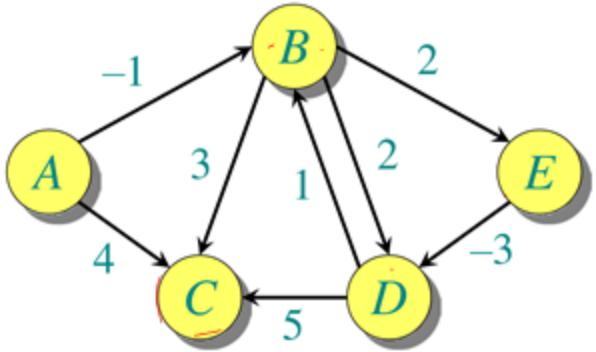
```
For each node v ∈ V
    M [0, v] = ∞
M [0, s] = 0
for i = 1 to n - 1
    for each node v ∈ V
        M [i, v] = M [i - 1, v]
        for each edge (u, v) ∈ E :
                M [i, v] ← min { M [i, v], M [i - 1, u] + w(u, v) }.
```

- Runtime: O(nm) = O(VE)
- Space Complexity: $O(n^2)$
  - Could be improved to O(n)
  - To compute M[i, v] only M[i-1,v] values are needed

Bellman-ford Algorithm: DP: example

s= A

	A	B	C	D	E
0	0	∞	∞	∞	∞
1	0	-1	4	∞	∞
2	0	-1	2	1	1
3	0	- 1	2	-2	1
4	0	-1	2	- 2	1



Bellman-ford Algorithm: dynamic programming approach

```
For each node v ∈ V
    M [0, v] = ∞
M [0, s] = 0
for i = 1 to n - 1
    for each node v ∈ V
        M [i, v] = M [i - 1, v]
        for each edge (u, v) ∈ E :
                M [i, v] ← min { M [i, v], M [i - 1, u] + w(u, v) }.
```

O(V)

O(V+E)

runtime = $O(V(V+E))$

$= O(V^2 + VE)$

$= O(VE)$

- Runtime: O(nm) = O(VE)
- Space Complexity: $O(n^2)$
  - Could be improved to O(n)
  - To compute M[i, v] only M[i-1,v] values are needed

Bellman-ford algorithm: simple version

The green additions convert this to an algm for negative cycle detection

```
For each node v ∈ V
        d[v] = ∞
d[s] = 0
for i = 1 to n – 1
        for each edge (u, v) ∈ E :
                If d[v] > d[u] + w(u,v)
                        d[v] ← d[u] + w(u, v)
                        parent[v] = u
```

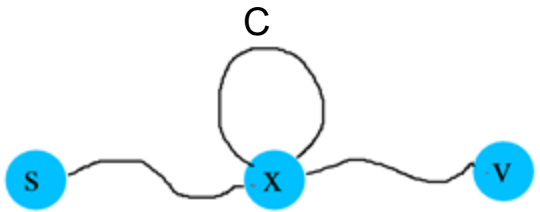
n

if i == n
there is a cycle

- Re-use same d[v]
- Runtime: O(nm)
- Space Complexity: O(n)
- The set of {v, parent[v]} form a shortest path tree
- Allows to recover path s to v backward from v
- How to detect negative weight cycle **reachable from s**
    - Run 1 more iteration and see if any d value changes

Algorithm for detecting Negative Cycles

- **Lemma**. If D(n, v) < D(n-1, v) for some v, then any shortest path from s to v contains a negative cycle.
- **Proof.** by contradiction.
  - Suppose G does not contains a negative cycle
  - Since D(n, v) < D(n-1, v), the shortest path from s to v has exactly n edges.
    - Otherwise, if a shortest path used at most n−1 edges, it would already be considered in D(n−1,v), implying D(n,v)=D(n−1,v) which is a contradiction.
  - By pigeonhole principle, a path of length n must have a repeated vertex, and thus a cycle c. We claim that C must be a negative cycle
  - If C has non-negative weight, removing it would give us a shortest path with less than n edges →contradiction: the path contained exactly n edges.
- there is a negative cycle. How do we find it?



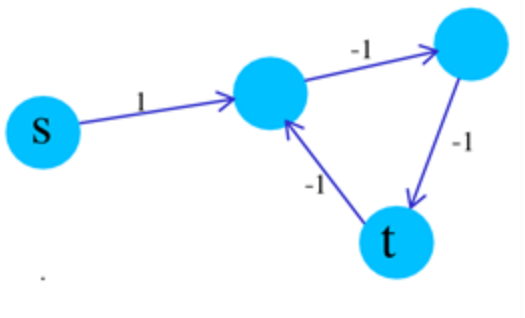
Algorithm for detecting Negative Cycles

- So, to detect a negative cycle reachable from s:
  - We run one more iteration, and check if any d value changes.
  - By tracing out the parents using the stored information, we can find P and thus the cycle C. This gives an O(mn) time algorithm to find a negative cycle, using theta(n^2) space

Negative Cycles

Claim 1. If there is a negative cycle on s → t path, then D(k, t) → -∞ as k → ∞ for some v ∈ V

Example: D(3, t) = -1, D(6, t) = -4, D(9, t) = -7



Negative Cycles

- A graph has no negative cycles reachable from s iff D(n, v) = D(n-1, v) for all v ∈ V
  - →There is an O(mn) algorithm for checking

- **Claim.** If the graph does not have any negative cycle, then D(n, v) = D(n-1, v) for all v ∈ V
  - **Proof**. Any cycle is non-negative, so we can assume that any shortest path from s to v has no cycle and thus it is of length at most n-1

- **Claim.** If D(n, v) = D(n-1, v) for all v ∈ V, then the graph has no negative cycles reachable form s
  - **Proof.** We can show that D(k, v) is finite when k goes to infinity for all v ∈ V
  - By claim 1, there are no negative cycles in graph